Unit - 1

Introduction :

Data structure can be defined as the group of data elements which provides an efficient way of storing and organising data in the computer, so that it can be used efficiently.

Basic Terminology

1. Data: Data can be defined as elementary value or a set of values.

Example :

34
12/01/2020
ISBN 81-203-088-02
III
Data Structures
Æ
250.00

Each value or collection of values can be termed as data.

2. Entity :

An entity is one which has certain attributes and these attributes have some values. from a domain.

Example : Student Employee

Attributes : Name : → Domain : (Any string)
DOB : → Domain : (Any positive integer)
Sex : → Domain : (M, F, others)
Address : → Domain : (Any string)

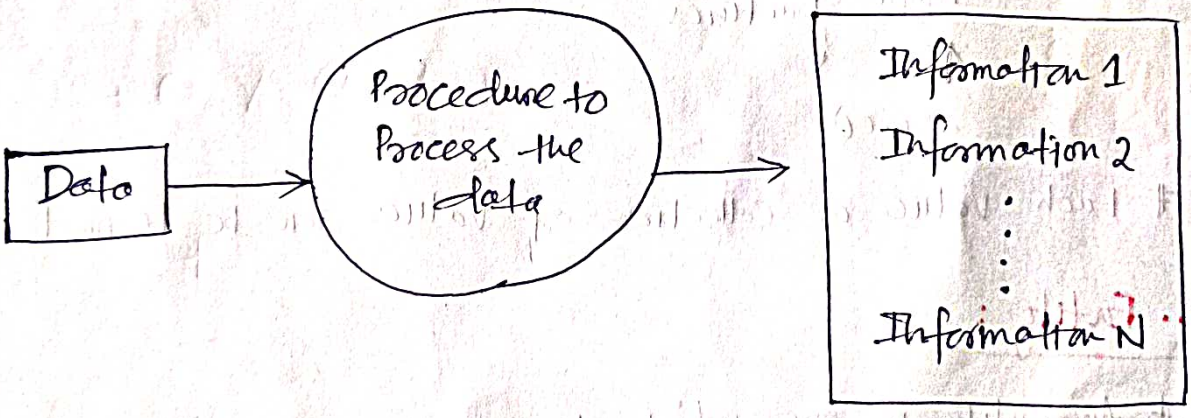
Domain of attributes is the set of all possible values that can be assigned to it.

3. Information:

Meaningful data or processed data is called information.

Example:

Data		Meaning
34	–	Age of a person
12/01/1988	–	Date of birth of a person.
ISBN-81-203-022-02	–	Book Number, recently published by the person
III	–	Number of award achieved by the person.
Data Structures	–	Name of the book
₹	–	Signature of the person
250.00	–	Price of the book



once we store data we can answer various questions related to the data. These answers are called information.

Example:

1. What is the age of the author?
2. What is the name of the book published by the author?

4. Datatypes:

Datatype is defined as set of values and operations that can be performed on that values.

Examples: Integer, Real, Boolean, Complex, Character.

Datastructure answers the following three questions.

1. How to store a value or values.
2. What are the operations on that type?
3. What amount of memory require to store values?

5. Abstract Datatypes:

The abstract datatype is a special kind of datatype, whose behaviour is defined by a set of values and set of operations.

Concept of data structures:

Manipulation of real-life data requires the following essential tasks.

1. Storage representation.
2. Retrival of stored data.
3. Transformation of user data.

For a given kind of user data, its structure implies the following.

1. Domain: This is the range of values that the data may have.
2. Function: This is the set of operations which may legally be applied to the element/data.
3. Axioms: This is the set of rules with which the different operation belongs to functions actually can be implimented.

A data structure is a triplet:

$$D = (\mathcal{O}, \mathcal{F}, \mathcal{A})$$

$\mathcal{O}$ = (0, ±1, ±2, ···)

$\mathcal{F}$ = (+, −, ×, /, %)

$\mathcal{A}$ = (A set of binary operations to perform addition, subtraction, division, multiplication and modulo operation)

The abstract data type Complex can be defined using the data structure Real as:

Complex z
{
  x : Real
  y : Real
}

$\mathcal{O}$ = Any real number.

$\mathcal{F} = (\oplus, \ominus, \otimes, \oslash, \nabla, ||)$

$\mathcal{A}$ = {

$z = z_1 \oplus z_2 = (x_1 + x_2) + i(y_1 + y_2)$ — Complex addition

$z = z_1 \ominus z_2 = (x_1 - x_2) + i(y_1 - y_2)$ — Complex subtraction

$z = z_1 \otimes z_2 = (x_1 \times x_2 - y_1 \times y_2) + i(x_1 \times y_2 + x_2 \times y_1)$ — Complex multiplication

$z = z_1 \oslash z_2 = \frac{x_1 \times x_2 + y_1 \times y_2}{x_2^2 + y_2^2} + i\frac{x_2 \times y_1 - x_1 \times y_2}{x_2^2 + y_2^2}$ — Complex division

$z = \nabla z_1 = \frac{x_1}{x_1^2 + y_1^2} - i\frac{y_1}{x_1^2 + y_1^2}$ — Complex Conjugate

$z = |z_1| = \sqrt{x_1^2 + y_1^2}$ — Complex magnitude.

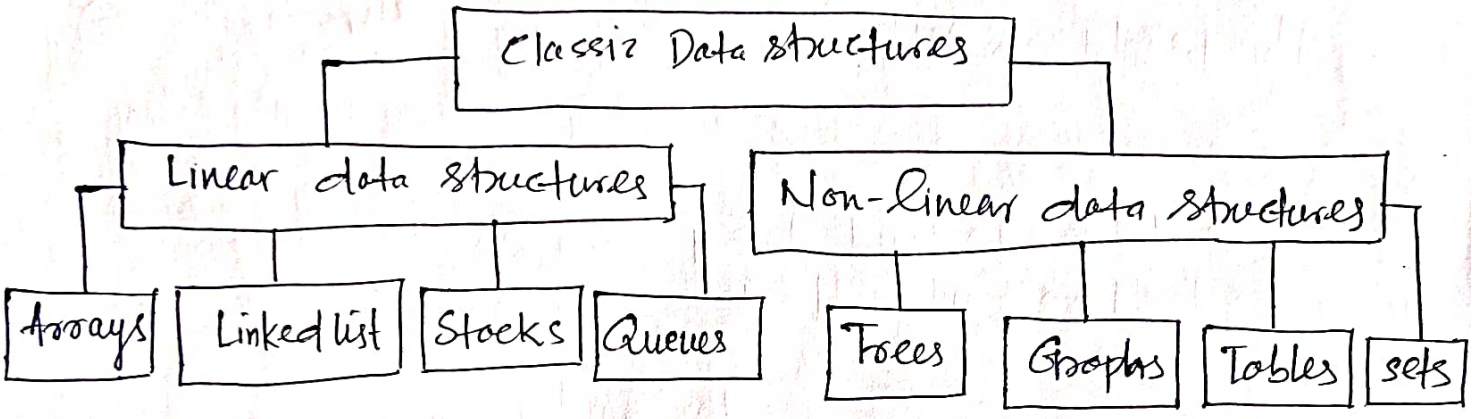
}

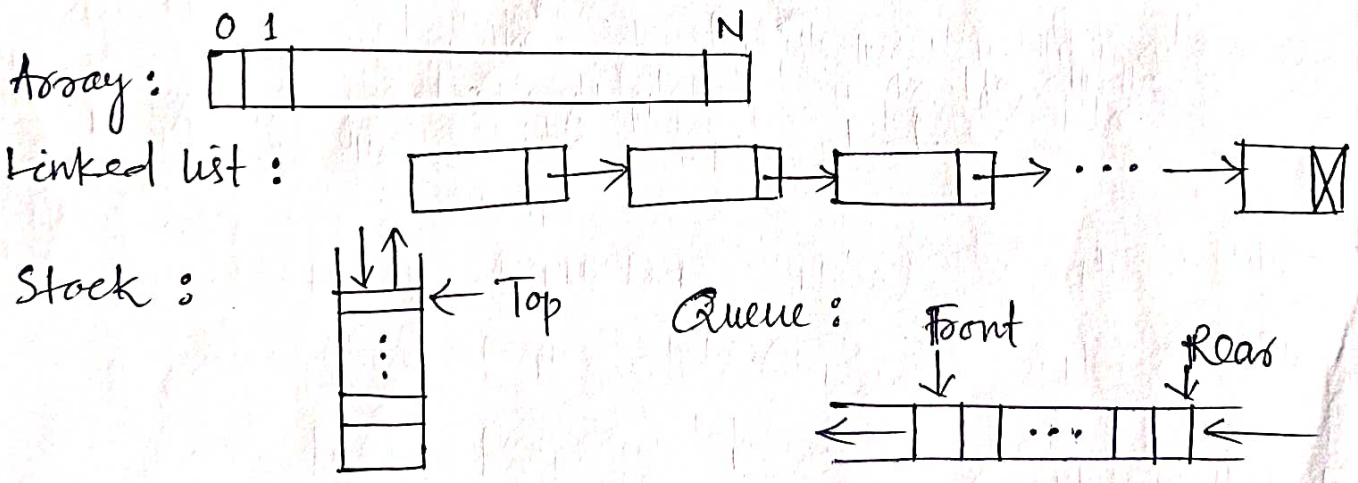
Overview of data structures:

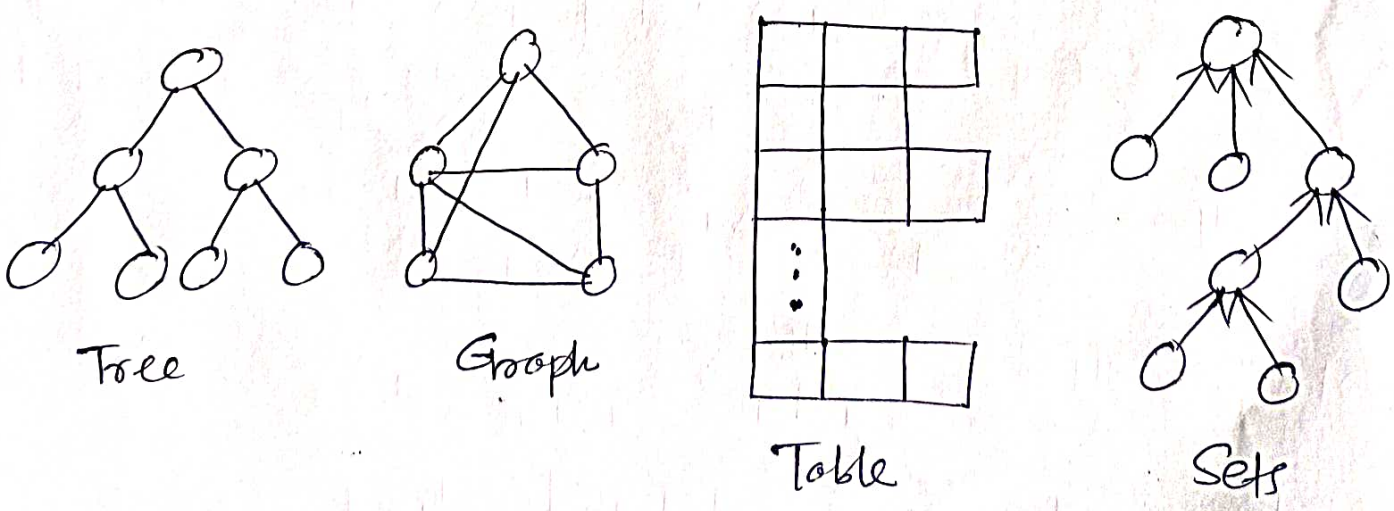
Classic data structures are classified into two main classes. Linear data structures and non linear data structures.

In linear data structures, all the elements form a sequence or maintain a linear ordering.

On the other hand, all the elements are distributed over a plane in case of non linear data structures.







Time and Space Complexity:

There is more than one way to solve a problem in Computer Science.

Therefore it is highly required to use a method to compare the solutions to judge the optimal solution.

These are two such methods used: time complexity and space complexity.

Time Complexity:

Time complexity quantifies the amount of time taken by an algorithm for the given input values.

Note that the time to run is a function of the length of the input not the actual execution time of the machine on which the algorithm is running.

It is assumed that a constant time c is taken to execute one operation, and total operation for an input length on N are calculated.

Example:

find whether a pair (X, Y) exists in an array A of N elements whose sum is Z.

```
int a[n];
for(int i=0; i<n; i++)
  cin>>a[i];
for(int i=0; i<n; i++)
    for(int j=0; j<n; j++)
      if(i!=j && a[i]+a[j]==z)
    return true;
return false;
```

During analyses of the algorithm, mostly the worst case scenario is considered. i.e when no pairs of elements with sum equals Z.

In the worst case:

1. N * c operations are required for input.
2. The outer loop i runs N times
3. For each i, the inner loop runs N times.

Total execution time is:

$N*C + N*N*C + C$;

Ignoring the lower order terms, since the lower order terms are relatively insignificant for large input, the highest order term N*N is taken without constant.

The worst case time will be represented as $O(N^2)$ So the time complexity is solely based on the number of elements in the array.

Space Complexity:

The space complexity of an algorithm quantifies the amount of space taken by an algorithm to run as a function of the length of the input.

Example:

Suppose a problem to find the frequency of array elements.

```
int freq[n];
int a[n];
for(int i=0; i<n; i++)
{     cin >> a[i];
}     freq[a[i]]++;
```

Here two arrays of length N, and variable i are used in the algorithm. So the total space used is

$$N * c + N * c + 1 * c = 2N * c + c$$

where c is a unit space taken.

For many inputs, constant c is insignificant. So space complexity is O(N).

Linked Lists :

Dynamic Memory Allocation :

Dynamic memory allocation is a request of executing program to the operating system to give it a block of main memory, so that it can use this memory for some purpose.

Usually the purpose is to add elements / nodes to a data structure.

C provides four library functions under stdlib.h header file to provide dynamic memory allocation. These are : 1. malloc(), 2. calloc(), 3. realloc, 4. free().

1. malloc() :

memory allocation.

malloc dynamically allocate a single large block of memory with specified size and returns a void pointer which can be typecast to any type.

Syntax : ptr = (cast-type *) malloc (byte-size)

Example : ptr = (int *) malloc (100 * sizeof(int));

2. calloc() :

Contiguous allocation.

calloc dynamically allocate the specified number of blocks of memory of the specified type.

It initializes each block with default value 0.

Syntax : ptr = (cast-type *) calloc (n, element-size);

Example : ptr = (float *) calloc (25, sizeof(float));

3. free():

- # free() method in C is used to dynamically de-allocate the memory.
- # memory allocated by malloc(), calloc() are not deallocated by their own.
- # Hence free() method is used, whenever dynamic memory allocation takes place.
- # Syntax: free(ptr);

4. realloc():

- # realloc() - is used to dynamically change the memory allocation of a previously allocated memory.
- # If memory allocation using malloc() or calloc() is insufficient, realloc() is used to dynamically re-allocate memory.

Syntax: ptr = realloc(ptr, new-size);

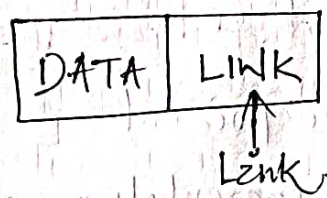
Linked Lists :

Linked list is a dynamic data structure where amount of memory required can be varied during its use.

In a linked list adjacency between the elements are maintained by means of links or pointers.

A link or pointer is the address of the subsequent element.

An element in a linked list is specifically termed as node and its structure is shown below :



Definition :

A linked list is an ordered collection of finite homogeneous data elements called nodes where linear order is maintained by means of links or pointers.

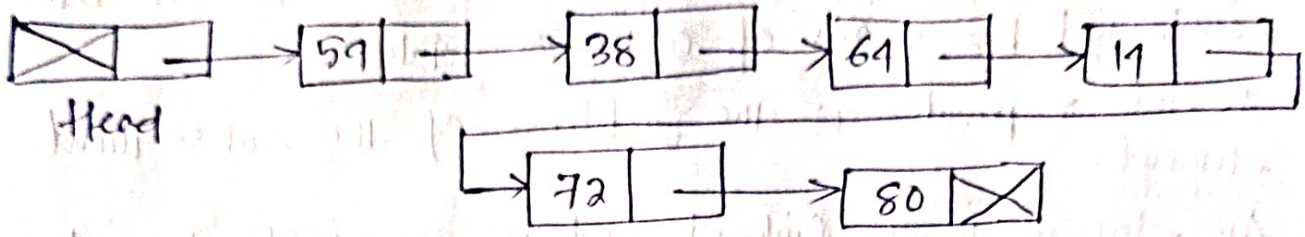
Depending on the requirements the pointers are maintained. Accordingly linked list can be classified into three major groups.

- Single linked list
- Circular linked list
- Double linked list.

① Single Link list :

In a single linked list each node contains only one link to the next node in the list.

Example :



A single link list with 6 nodes.

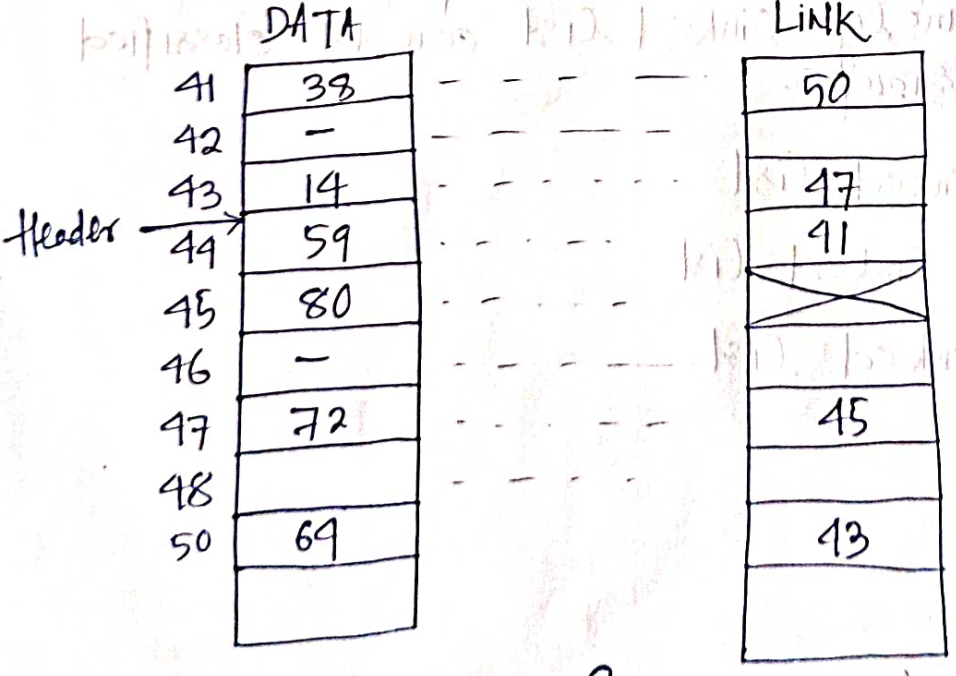
Representation of a linked list in the memory :

There are two ways to represent a linked list in memory.

1. Static representation in memory
2. Dynamic representation in memory.

1. Static Representation :

Static representation of a single linked list maintains two arrays. One for data and another for links.



(Array of pointers)

2. Dynamic Representation:

In dynamic representation whenever a node is required the memory is allocated from the heap by using dynamic allocation.
A linked list is represented by a pointer to the first node of the linked list. The first node is called head.
If the linked list is empty the value of the head is NULL.
Each node consists of two parts.
    1. Data
    2. Pointer to the next node.
In C we represent a node using structures.

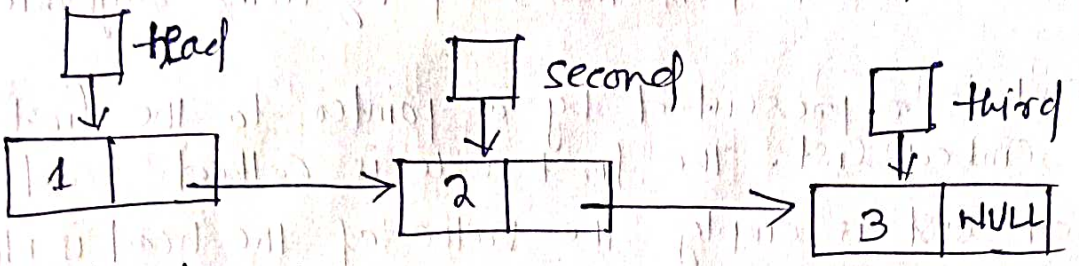
```
struct Node
{
    int data;
    struct Node * next;
};
```

A simple program to introduce a linked list:

```
#include<stdio.h>
#include<stdlib.h>
struct Node
{   int data;
    struct Node * next;
};
int main()
{
    struct Node * head = NULL;
    struct Node * second = NULL;
    struct Node * third = NULL;
```

```
void printlist(struct Node *p)
{
    while(n != NULL)
    {
        printf(" %d", n->data);
        n = n-> next;
    }
}
```

```
head = (struct Node *) malloc (sizeof (struct Node));
second = (struct Node *) malloc (sizeof (struct Node));
third = (struct Node *) malloc (sizeof (struct Node));
```




```
head -> data = 1;
head -> next = second;
second -> data = 2;
second -> next = third;
third -> data = 3;
third -> next = NULL;

printlist (head);

return 0;
}
```

Operations on a single linked list :

Possible operations on a single linked list are :

1. Insertion of a node
2. Deletion of a node
3. Searching of an element
4. Traversing a linked list.

1. Insertion of a node :

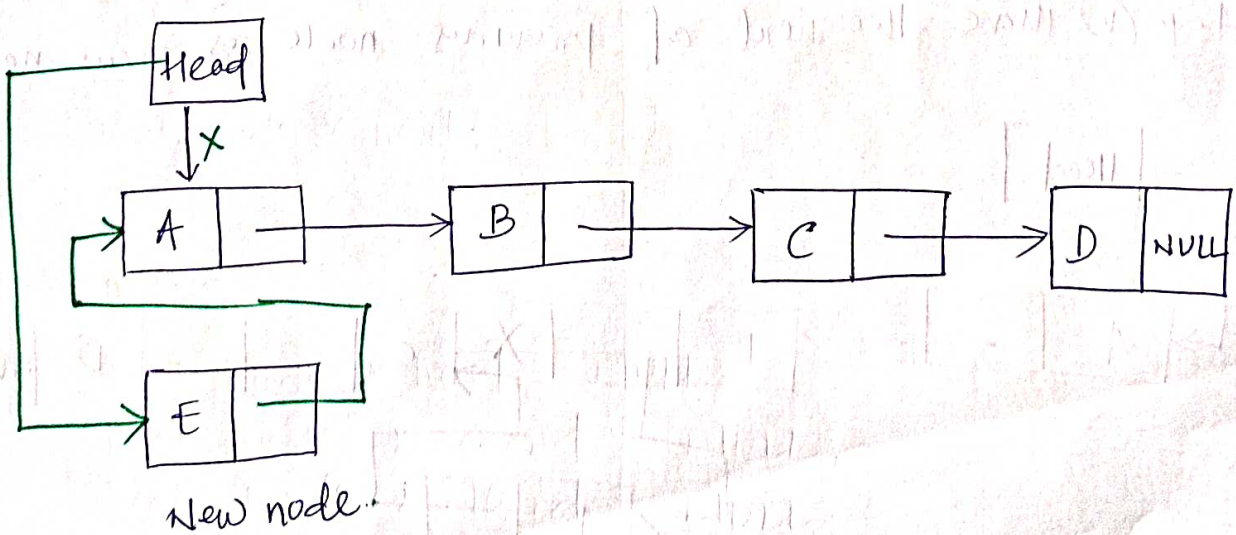
There are various positions where a node can be inserted.

i. Insertion at front.
ii. Insertion at end.
iii Insertion at any position.

i. Insertion of a node at the front :

This is a four step process :

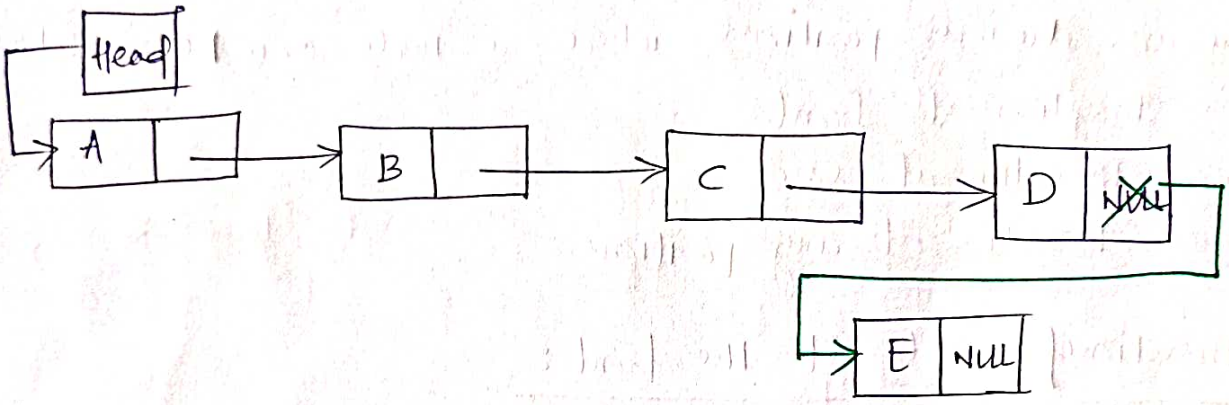
Step-(I) Allocate memory for new node.
Step-(II) put the data in the newly allocated node.
Step-(III) Make next of new node to point the starting (head) of the linked list.
Step-(IV) Make the head to point the new node.



(ii.) Insert a node at the end:

This is a six step process:

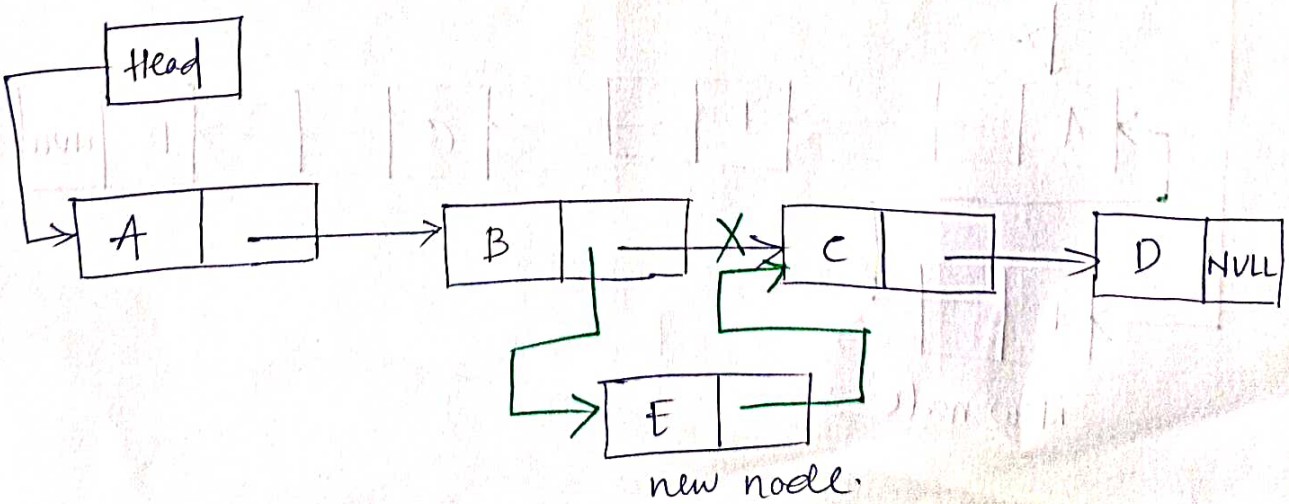
step (I) Allocate the memory for new node.
step (II) Put the data in the node.
step (III) Make next of new node to NULL
step (IV) If linked list is empty make new node as head node.
step (V) Else traverse till the last node.
step (VI) change the next of last node to new node.



(iii.) Add a node after a given node:

This is a five step process:

step (I) Check if given node is null
step (II) Allocate a new node.
step (III) Put the data in new node.
step (IV) make next of new node as next of previous node.
step (V) Move the next of previous node as new node.



(9)

Complete Program to demonstrate all insertion operation:

```
#include<stdio.h>
#include<stdlib.h>
struct Node {
    int data;
    struct Node * next;
};
void printList (struct Node *n)
{
    while (n != NULL)
    {
        printf (" %d", n->data);
        n = n->next;
    }
}
void insertAtFront(struct Node** head_ref, int new_data)
{
1. struct Node *new_node = (struct Node *) malloc (sizeof(struct Node));
2. new_node->data = new_data;
3. new_node->next = (*head_ref);
4. (*head_ref) = new_node;
}
void insertAtEnd(struct Node ** head_ref, int new_data)
{
1. struct Node *new_node = (struct Node*) malloc(sizeof(struct Node));
   struct Node *last = *head_ref;
2. new_node->data = new_data;
3. new_node->next = NULL;
4. if (*head_ref == NULL)
   {
       *head_ref = new_node;
       return;
   }
```

```
5. while (last->next != NULL)
       last = last->next;
6. last->next = new_node;
   return;
}
void insertAfter(struct Node* prev_node, int new_data)
{
1. if (prev_node == NULL)
   {  printf("the given previous node cannot be NULL");
      return;
   }
2. struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));
3. new_node->data = new_data;
4. new_node->next = prev_node->next;
5. prev_node->next = new_node;
}
int main()
{  struct Node* head = NULL;
   insertAtEnd(&head, 6);
   insertAtfront(&head, 7);
   insertAtfront(&head, 1);
   insertAtEnd(&head, 4);
   insertAfter(head->next, 8);
   printf("\n Created linked list is:");
   printList(head);
   return 0;
}
```

Output :

Created linked list is: 1 7 8 6 4

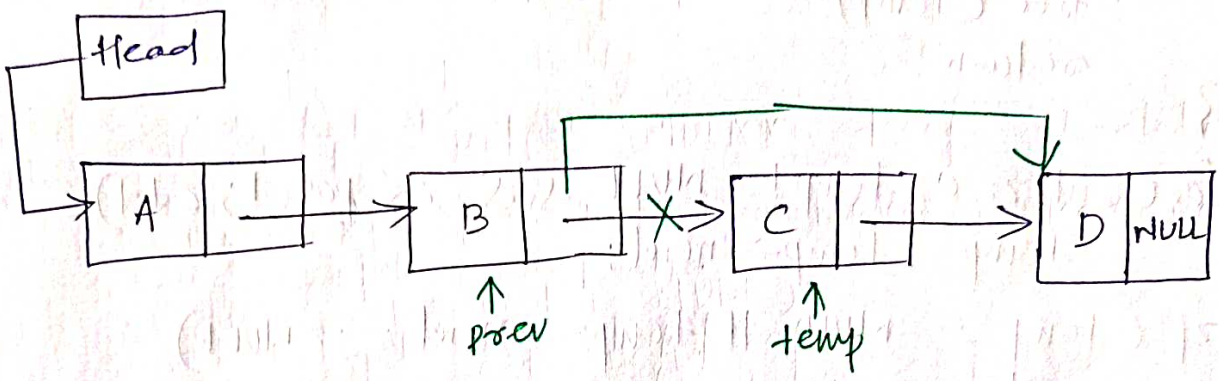
2. Deletion of a Node :

To delete a node from linked list we need to do the following:

step (i) Find the previous node to be deleted

step (ii) change the next of previous node.

step (iii) Free the memory to be deleted.




```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
void insert (struct Node **head_ref, int new_data)
{
    struct Node * new_data = (struct Node *)
        malloc (sizeof (struct Node));
    new_node -> data = new_data;
    new_node -> next = (*head_ref);
    (*head_ref) = new_node;
}
```

```
void delete_position(struct node **head_ref, int pos)
{
    if(*head_ref == NULL)
        return;
    struct Node *temp = *head_ref;
    if (pos == 0)
    {
        *head_ref = temp->next;
        free (temp);
        return;
    }
    for(int i=0; temp != NULL && i< pos-1; i++)
        temp = temp->next;
    if (temp == NULL || temp->next == NULL)
        return;
    struct Node *next = temp->next->next;
    free (temp->next);
    temp->next = next;
}

void delete_key (struct Node **head_ref, int key)
{
    struct Node *temp = *head_ref;
    struct Node * prev;
    if (temp != NULL && temp->data == key)
    {
        *head_ref = temp->next;
        free (temp);
        return;
    }
    while (temp != NULL && temp->data != key)
    {
        prev =temp;
        temp = temp->next;
    }
    if (temp== NULL) return;
    prev->next = temp->next;
    free (temp);
}
```

```
int main()
{
    struct Node *head = NULL;
    insert(&head,7);
    insert(&head,1);
    insert(&head,3);
    insert(&head,2);
    insert(&head,8);
    printf("Created linked list");
    printlist(head);
```

8 2 3 1 7

```
    delete_pos(&head,4)
    printlist(head)
```

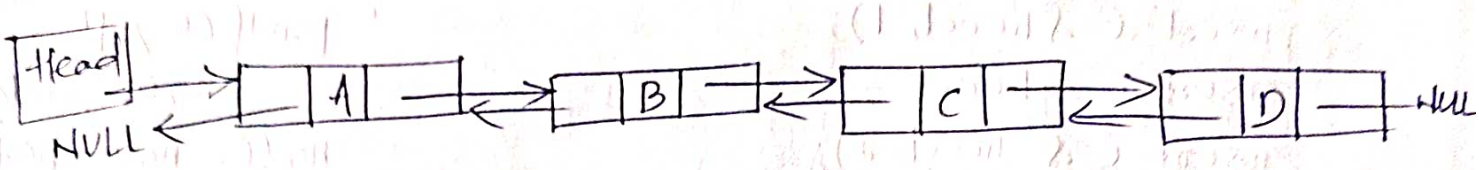
8 2 3 1

```
    delete_key(&head,3);
    printlist(head);
    return 0;
}
```

```
void printlist(struct Node* node)
{
    while(node!=NULL)
    {
        printf("%d", node->data);
        node = node->next;
    }
}
```

Doubly Linked List:

A Doubly linked list (DLL) contains an extra pointer typically called previous pointer together with next pointer and data.



The general structure of a node is:

```
struct Node {
        int data;
        struct Node * next;
        struct Node * prev;
};
```

Advantage over single linked list:

1. Can be traversed in both forward and backward direction
2. The delete operation in DLL is more efficient.
3. Insertion can be done quickly.

Disadvantage of Doubly linked list:

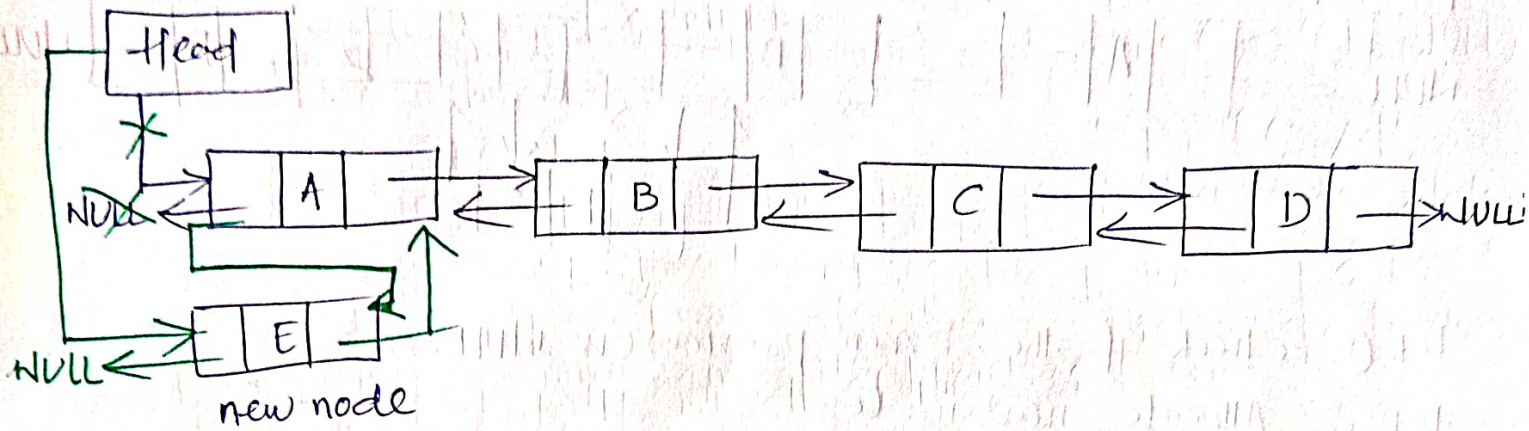
1. Every node requires extra space for previous pointer.
2. All operations require an extra pointer previous to be maintained.

Insertion in DLL

Similar to single link list insertion can be done in three ways in DLL.

1. At the front of a DLL
2. After a given node.
3. At the end of the DLL

(1) At the front of DLL :



This is ~~five~~ step process :

step-(1) Allocate memory for the new node

step-(2) put the data in the new node.

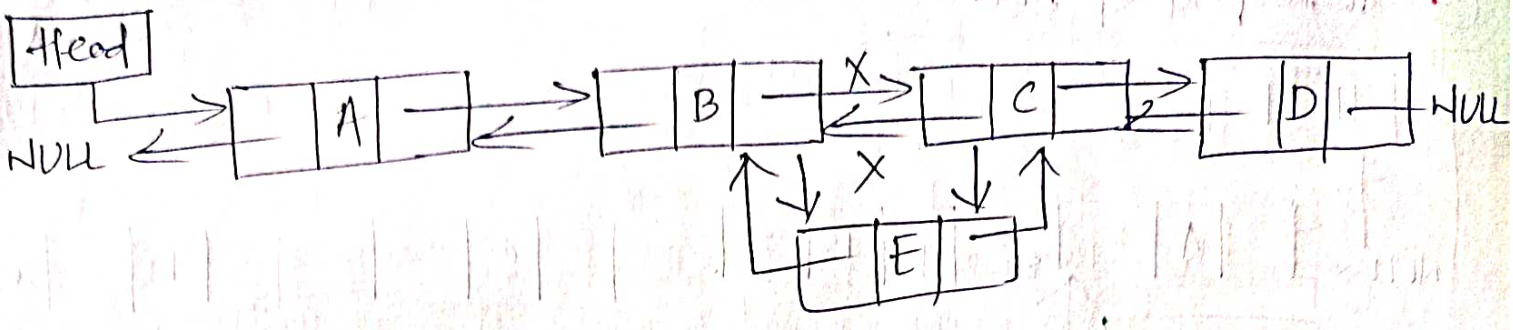
step-(3) make next of new node as head and previous as NULL.

step-(4) change prev of head node to new node.

step-(5) Move the head to point to the new node.

```
void insert_at_front(struct Node **head_ref, int new_data)
{ struct Node *new_node = (struct Node *)malloc(
                          sizeof(struct Node));
    new_node -> data = new_data;
    new_node -> next = (*head_ref);
    new_node -> prev = NULL;
    if ((*head_ref) != NULL)
        (*head_ref) -> prev = new_node;
    (*head_ref) = new_node;
```

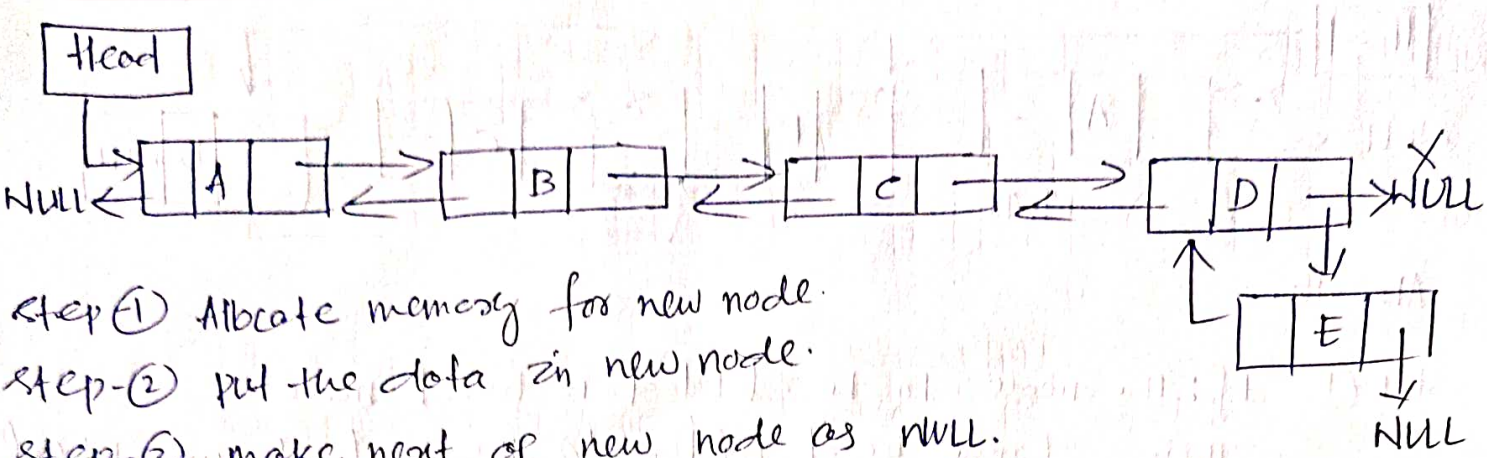
(2) Insertion after a given node:



Step (1) Check if the given previous is NULL.
step-(2) Allocate new node.
step-(3) put data in new node.
step-(4) Make the next of new node as next of previous node.
step-(5) Make the next of prev_node as new node.
step-(6) make prev_node as previous of new node.
step-(7) change previous of new node's next node as newnode.

```
void insertafter (struct Node * prev_node, int new_data)
{
    if (prev_node == NULL)
    { printf ("The given previous node cannot be NULL");
      return;
    }
    struct Node * new_node = (struct Node *)
          malloc (sizeof (struct Node));
    new_node -> data = new_data;
    new_node -> next = prev_node -> next;
    prev_node -> next = new_node;
    new_node -> prev = prev_node;
    if (new_node -> next != NULL)
          new_node -> next -> prev = new_node;
}
```

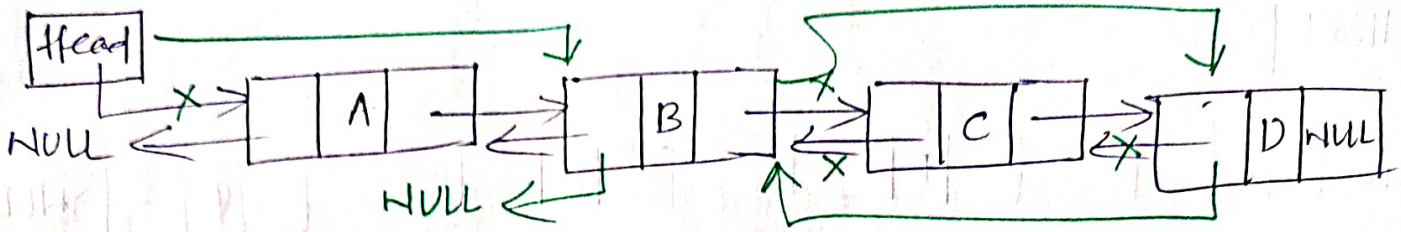
3. Insertion at the end :



Step ① Allocate memory for new node.
Step-② put the data in new node.
Step-③ make next of new node as NULL.
Step-④ If the linked list is empty then make the new node as head.
Step-⑤ Else traverse till the last node.
Step-⑥ Change the next of last node as new node.
Step-⑦ make last node as previous of new node.

```
void insert_at_end(struct Node **head_ref, int new_data)
{  struct Node * new_node = (struct Node *)
              malloc (sizeof (struct Node));
   struct Node * last = *head_ref;
   new_node -> data = new_data;
   new_node -> next = NULL;
   if (*head_ref == NULL)
   {    new_node -> prev = NULL;
        *head_ref = new_node;
        return;
   }
   while (last -> next != NULL)
              last = last -> next;
   last -> next = new_node;
   new_node -> prev = last;
   return;
}
```

Deletion in Doubly Linked list :



Algorithm :

Step ① If : the node to be deleted is head node.
- Then change head pointer to the next of current head.
- make previous of current next node to NULL
- free the first node.

Step - ② Else :
If the current node is not the last node
Then change next's previous to current previous.
Else
change current previous's next to current next.

Step - ③ finally free the memory of current node.

```
void deletenode(struct Node **head_ref, struct Node* curr)
{
    if (*head_ref == NULL || curr == NULL)
        return;
    if (*head_ref == curr)
        *head_ref = curr->next;
    if (curr->next != NULL)
        curr->next->prev = curr->prev;
    if (curr->prev != NULL)
        curr->prev->next = curr->next;
        free (curr);
    return ;
}
```

Delete a node from a particular position:

Algorithm:

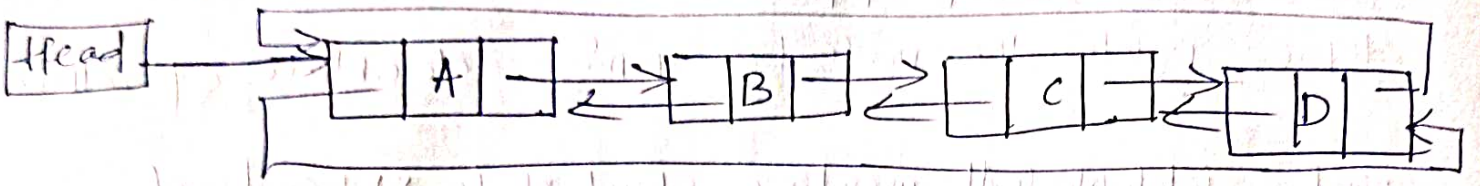
Step ① Goto position of the given node by traversing the doubly linked list.

Step-② Delete the node by using deleteNode operation.

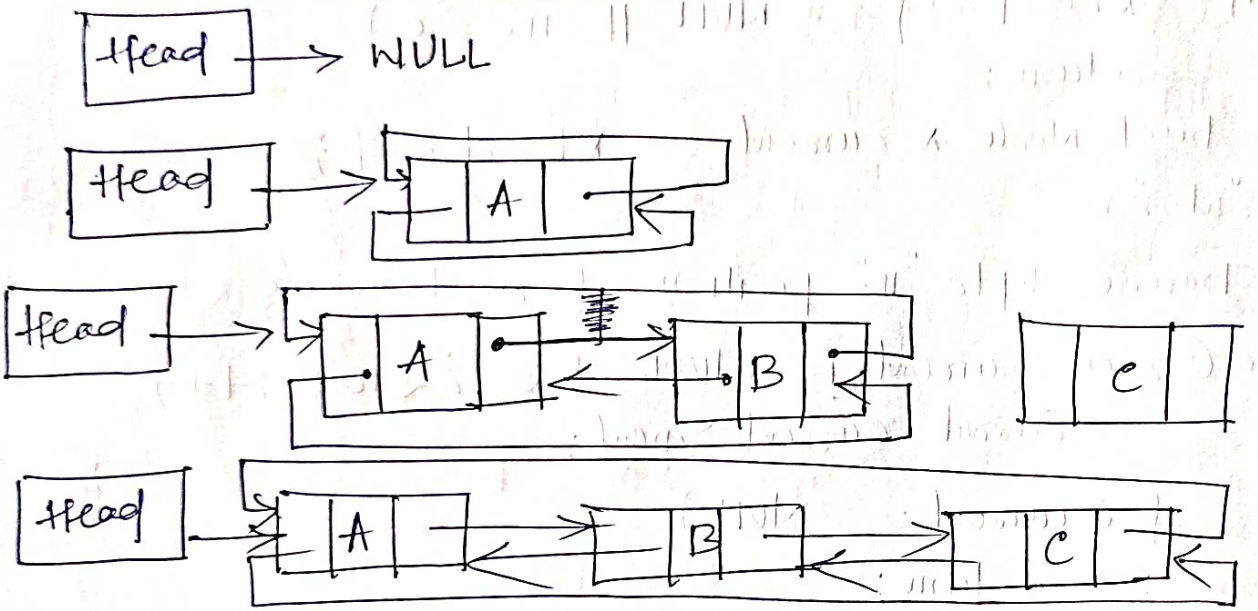
```
void deleteNodeAtGivenPos(struct Node **head_ref,
                          int pos)
{
    if (*head_ref == NULL || n<=0)
        return;
    struct Node * current = *head_ref;
    int i;
    // Traverse upto "n" position from beginning.
    for(i=0; current != NULL && i<n; i++)
        current= current->next;
    if (current == NULL)
        return;
    deleteNode(head_ref, current);
}
```

Doubly Circular Linked list :

In circular doubly linked list the last node points to first node by next pointer and also first node points to last node by previous pointer.



Insertion in Circular Double Linked List :



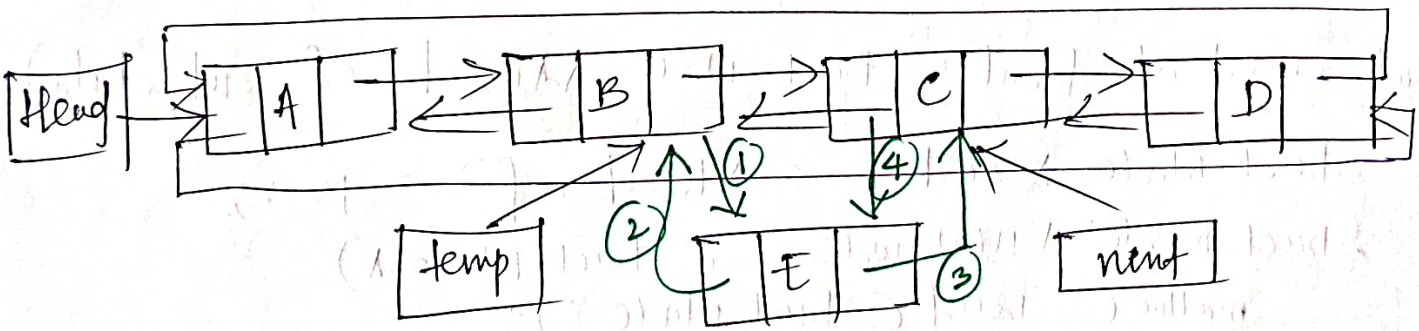
Insertion at the end :

Step-① If the list is empty create a single node.
Step-② Else find the last node.
Step-③ create the new node and assign the data.
Step-④ Make the new_node next to head.
Step-⑤ make prev of head to new_node.
Step-⑥ make new node previous to last node.
Step-⑦ make last node next to new node.

```
void insert_at_end (struct Node **head_ref, int data)
{
    if (*head_ref == NULL)
    {
        struct Node * new_node = (struct Node *)
            malloc (sizeof (struct Node));
        new_node -> data = data;
        new_node -> next = new_node -> prev;
        new_node -> prev = new_node;
        *head_ref = new_node;
        return;
    }
    struct Node *last = (*head_ref) -> prev;
    struct Node *new_node = new_node -> data = data;
     new_node -> next = *head_ref;
     (*head_ref) -> prev = new_node;
      new_node -> prev = last;
       last -> next = new_node;
       return;
}
void insert_at_front (struct Node **head_ref, int data)
{  struct Node * last = (*head_ref) -> prev;
   struct Node *new_node = (struct Node *)
       malloc (sizeof (struct Node));
    new_node -> data = value;
    new_node -> next = *head_ref;
    new_node -> prev = last;
     last -> next = new_node;
     (*head_ref) -> prev = new_node;
       *head = new_node;
}
```

Function to insert with value1 after the value2 :

```
void insert_after(struct Node **head_ref, int value1,
                  int value2)
{
    struct Node *new_node = (struct Node *)
        malloc(sizeof(struct Node));
    new_node->data = value1;
    struct Node *temp = *head_ref;
    while (temp->data != value2)
        temp = temp->next;
    struct Node *next = temp->next;
    temp->next = new_node;
    new_node->prev = temp;
    new_node->next = next;
    next->prev = new_node;
}
```



Sparse Matrices :

If most of the elements of a matrix have value 0, then it is called a sparse matrix.

Sparse matrix are used because of less storage requirement compared to array representation.

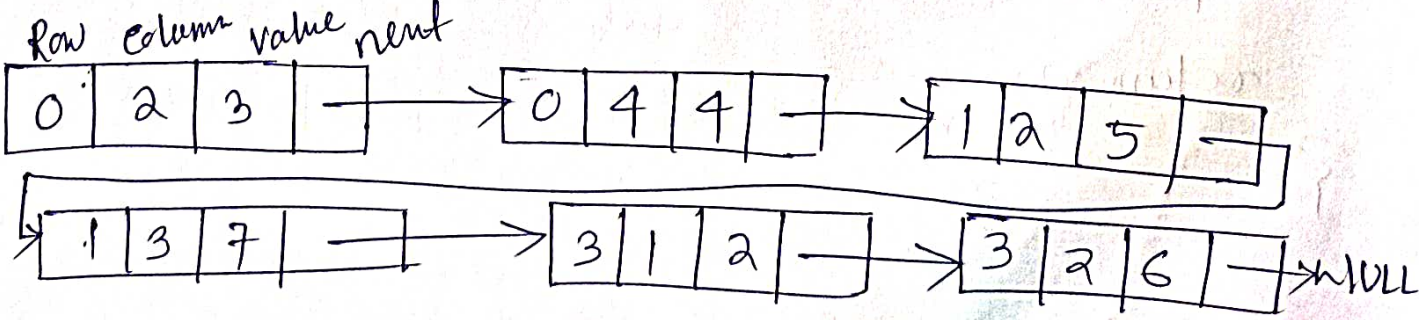
Example :

$$A = \begin{bmatrix} 0 & 0 & 3 & 0 & 4 \\ 0 & 0 & 5 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 6 & 0 & 0 \end{bmatrix}$$

```
int Matrix[4][5] = { {0,0,3,0,4},
                     {0,0,5,7,0},
                     {0,0,0,0,0},
                     {0,2,6,0,0} };
```

To represent a sparse matrix as linked list each node has four fields.

Row : Index of row where non-zero element is located.
Column : Index of column where non-zero element is located.
value : Value of non-zero element.
Next Node : Address of next node.



Program to implement Sparse matrix :

```
#include<stdio.h>
#include<stdlib.h>

struct Node
{   int value;
    int row;
    int col;
    struct Node* next;
};
void create_node(struct Node **start, int data, int r, int c);
void print(struct Node *start);

int main()
{   int smatrix[4][5] = { {0,0,9,0,4},
                          {0,0,5,7,0},
                          {0,0,0,0},
                          {0,2,6,0,0} };
    int i, j;
    struct Node *start = NULL;
    for(i=0; i<4; i++)
        for(j=0; j<5; j++)
            if(smatrix[i][j] != 0)
                create_node(&start, smatrix[i][j], i, j);
    print(start);

    return 0;
}
```

```
void create_node (struct Node **start, int v, int r, int c)
{
    struct Node *temp, *curr;
    temp = *start;
    if (temp == NULL)
    {
        temp = (struct Node *) malloc (sizeof (struct Node));
        temp -> value = v;
        temp -> row = r;
        temp -> col = c;
        temp -> next = NULL;
        *start = temp;
    }
    else {
        while (temp -> next != NULL)
            temp = temp -> next;
        curr = (struct Node *) malloc (sizeof (struct Node));
            curr -> value = v;
            curr -> row = r;
            curr -> col = c;
            curr -> next = NULL;
            temp -> next = curr;
    }
}
void print (struct Node *start)
{
    struct Node *temp;
        temp = start;
    while (temp != NULL)
    { printf ("row = %d, col = %d, value = %d",
        temp -> row, temp -> col, temp -> value);
        temp = temp -> next;
    }
    printf ("\n");
}
```